OBELIX: the Warehouse Robot
CS780 Capstone Project

Aritra Ambudh Dutta
230191
aritraad23@iitk.ac.in
Indian Institute of Technology Kanpur

Abstract

This report explores reinforcement learning for the OBELIX warehouse robot, which must find, attach to, and push a grey box to the arena boundary in a Partially Observable Markov Decision Process (POMDP) with 18-bit binary sensors. Over four phases, majorly seven approaches were investigated—DQN, Dueling D3QN with GRU, Recurrent PPO, Discrete SAC, and hybrid heuristic-RL—employing potential-based reward shaping, curriculum learning, frame stacking, and prioritized experience replay. Sparse rewards and perceptual aliasing caused persistent circling failures across most methods. The best agent, **Recurrent PPO with LSTM**, scored **-1447.55** on the Codabench final test phase.

1 Competition Result

Codabench Username: a_230191
Final leaderboard rank (Test Phase): 11
Leaderboard rank – Level 1 (static): 39
Leaderboard rank – Level 2 (blinking): 80
Leaderboard rank – Level 3 (moving + blinking): 48

Number of valid submissions made:

Phase	Submissions
Development – Level 1 (Static)	15
Development – Level 2 (Blinking)	11
Development – Level 3 (Moving + Blinking)	53
Final Test Phase	366

Table 1: Number of valid submissions per phase.

2 Introduction and Problem Description

The OBELIX project draws inspiration from the seminal 1991 work by Mahadevan and Connell [1], which demonstrated one of the earliest applications of reinforcement learning on a physical robot for box-pushing tasks. The task is to design an RL-based autonomous controller for a simulated mobile robot named OBELIX that must locate a grey box in an arena, attach to it, and push it to the arena boundary (success).

The environment presents a **Partially Observable Markov Decision Process (POMDP)** with the following characteristics:

- **Observations**: An 18-bit binary vector consisting of 8 near sonar sensors, 8 far sonar sensors, 1 infrared (IR) sensor for direct frontal detection, and 1 indicator showing whether the robot is stuck against a wall or boundary.
- **Actions**: 5 discrete actions – rotate left 45° (L45), rotate left 22.5° (L22), move forward (FW), rotate right 22.5° (R22), and rotate right 45° (R45).

- **Rewards**: Sparse and one-time (per-episode) sensor rewards (+1 for side sensors, +2 for far forward, +3 for near forward, +5 for IR), a large box attachment bonus (+100), push penalty (−1/step while pushing), huge success bonus (+2000), wall collision penalty (−200), and idle penalty (−1/step with no sensor activated).

The task is evaluated across **three** difficulty levels of increasing complexity:

1. **Level 1: Static Box**: The box remains stationary. The robot navigates to it, attaches, and pushes to the boundary.
2. **Level 2: Blinking Box**: The box randomly appears and disappears. The robot cannot attach when the box is invisible, but once attached, blinking stops.
3. **Level 3: Moving + Blinking Box**: The box moves with constant velocity and also blinks. Movement stops once the robot attaches.

Additionally, a **vertical wall with a small opening** may appear in any level, introducing navigation obstacles with a harsh −200 penalty for wall collisions. The evaluation uses hidden random seeds across 10 runs per difficulty level, making overfitting to specific configurations infeasible.

The core challenges are: (1) **perceptual aliasing**: Under partial observability, many world states map to identical observations (especially all-zeros when the box is out of range); (2) **sparse rewards**: randomly finding, attaching, and pushing the box is highly unlikely; and (3) **exploration-exploitation trade-off**: systematic exploration when no informative sensors are active, and efficient exploitation once sensor cues become available. The goal was to maximize **mean cumulative reward** across all levels with wall robustness.

3 Methodology and Model Evolution

My work evolved significantly across the four development phases, with each iteration informed by the failures and insights of prior attempts.

3.1 Phase 1–2: Initial Exploration (Levels 1 and 2)

I explored six baseline algorithms: **(a) Standard DQN** (18→64→64→5 MLP, 1K–2K episodes): consistently converged to a degenerate circling policy (L22↔R22), scoring ≈−1980; **(b) Tabular Q-Learning** (2^{18} = 262K states, up to 1M episodes): slow convergence due to enormous state space; **(c) A2C** (shared actor-critic, 5K episodes): marginally better exploration but still failed to find the box; **(d) Dueling DRQN** (GRU-based, 18→64→GRU(64)→5, 4-frame stack): had memory capability but still circled; **(e) DRQN with Anti-Rotation Reward Shaping** (−0.2 anti-rotation penalty after sustained turn streaks, with an additional −0.5 penalty under prolonged same-observation turning; +1.5 FW bonus on zero sensors to discourage Rotation; 72-dim input through a 16-step GRU): theoritically best Phase 1–2 approach but still scored −1980 on Codabench; **(f) Dueling DDQN with Memory Augmentation** (22-dim input: 18 raw + visibility flag, steps-since-seen, sin/cos angle to box; 128-unit shared layers with dueling heads): could have helped when the box was blinking, but still could not solve the main challenge of reliably finding the box.

3.1.1 Root Cause Analysis

After Phase 1–2, I identified four reasons for the persistent circling behavior: (1) **Perceptual aliasing**: many world states map to identical observations, especially the all-zeros case; (2) **Deceptive rewards**: Positive rewards are sparse and hard to reach, while random forward exploration often causes immediate penalties (e.g., *collisions/idle loss*), so the agent finds repetitive turning safer than risky movement toward true task completion; (3) **Exploration failure**: ϵ-greedy cannot reliably find the box; (4) **Shaping distortion**: aggressive bonuses caused optimization for shaped rather than true rewards.

3.2 Phase 3: Structured Approaches (Level 3)

Based on the analysis, I developed four structured approaches:

Approach 1 – Recurrent PPO (RPPO) [2]: LSTM-based actor-critic (77-dim input: 4-frame stack + one-hot prev-action, 128-unit LSTM). Curriculum learning over 5 phases (spawn distance: 20px → random) with 7-component dense reward shaping. Performed well in training but poorly on Codabench's random spawns (−1500 to −4000).

Approach 2 – Dueling D3QN + GRU + PER [3, 4, 5]: 128-unit GRU with Prioritized Experience Replay (PER) (α=0.6, β: 0.4→1.0 via SumTree) and 3-step returns. Same input/curriculum as Approach 1. Best single-episode score in Phase 3 (−1515) but inconsistent.

Approach 3 – Discrete SAC [6]: Entropy-regularized $J(\pi) = \mathbb{E}[Q(s,a) - \alpha \log \pi(a|s)]$ with twin GRU-based Q-networks and auto-tuned temperature (α), to encourage exploration and reduce premature collapse to circling policies. In my Phase 3 runs, it did not converge within the available training budget.

Approach 4 – Hybrid Heuristic + DQN: Mode-switch hybrid controller with hand-crafted search/stuck-escape heuristics and a small DQN (77→64→64→5) for sensor-active approach/push decisions (with heuristic–NN blending). Practical for reducing random wandering, but still brittle with wall obstacles.

3.3 Phase 4: Final Approaches (Test Phase)

Phase 3 established the core design logic, but its results were **limited by both compute and tuning**: the runs were configured with max_steps = 1000, intended 8K–10K episodes, and relatively large replay settings, yet most experiments had to stop early (typically around ∼2K episodes) due to limited compute. In addition, these settings were not always well matched to the task (for *long-horizon sparse-reward exploration*), so Phase 3 outcomes should be interpreted as ***under-trained or suboptimally tuned*** rather than definitive failures of the underlying ideas.

In Phase 4, I kept the previous Phase 3 approaches for continuity and fair re-checking, but introduced **three new methods** (Approaches 5–7), including **no-curriculum variants** (Approaches 5 and 6) motivated directly by the Phase 3 shortcomings. Thus, the final iteration covered **7 total approaches** (4 retained + 3 new). The Phase 4 setups used **tuned hyperparameters with sufficient compute** (typically 10K–15K episodes, max_steps = 2000, replay sizes about 50K–75K), which improved stability and yielded stronger Codabench results.

The practical reason these changes helped is that **longer episodes with higher max_steps** exposed the full search→approach→push trajectory more often, while **moderate replay sizes** reduced the dominance of stale early transitions from weak policies. Together, this made *training updates less noisy under sparse rewards* and made optimization less noisy than the short-horizon, over-large-buffer Phase 3 setting.
This led to three refined approaches:

Approach 5 – Simple DDQN without Curriculum: I used a 3-layer MLP (two hidden layers: $77 \to 128 \to 128 \to 5$) Double DQN with **no curriculum learning** and **minimal reward shaping** (only anti-circling: -3.0 for 6+ consecutive turns, and anti-stuck: -10.0). The key innovation was a **heuristic fallback/search policy** at inference: when all sensors are zero, instead of relying on NN outputs, the agent follows a rule-based "*lawnmower*" pattern (15 forward steps, then a 90° turn via two 45° turns, alternating direction). Training used mixed difficulty sampling per episode (0/2/3), with wall obstacles always enabled, for 15,000 episodes.

Approach 6 – D3QN v2 with GRU, No Curriculum: Similar philosophy of minimal intervention applied to the recurrent architecture. Dueling D3QN with 128-unit GRU, 77-dim input, 16-step sequences. No curriculum, minimal shaping. The GRU hidden state was reset after 5 consecutive zero-observations to prevent saturation. Trained for 12,000 episodes across mixed difficulties with $wall_obstacles = True$ always.

Approach 7 – Refined D3QN with PBRS: My last approach used Potential-Based Reward Shaping (PBRS) [7], which is theoretically guaranteed to preserve optimal policies. Three shaping components were used:

$$\Phi_{\text{approach}}(s) = -\frac{d(\text{robot}, \text{box})}{d_{\max}} \tag{1}$$

$$\Phi_{\text{heading}}(s) = \cos(\theta_{\text{facing}} - \theta_{\text{to_box}}) \quad \text{(pre-attachment only)} \tag{2}$$

$$\Phi_{\text{push}}(s) = -\frac{d(\text{box, nearest_wall})}{d_{\max}} \quad \text{(post-attachment only)} \tag{3}$$

The shaped reward at each step followed $F(s, s') = \gamma\Phi(s') - \Phi(s)$ with a global shaping scale of 10.0; in implementation, heading and push terms were additionally weighted by 0.5 and 2.0 respectively. The architecture used a larger Dueling Double DQN (shared encoder: $108 \to 256 \to 128$ via 6-frame stacking), with PER ($\alpha = 0.6$) and 3-step returns. Curriculum was reintroduced conservatively in four phases (no-wall warm-up → walls → blinking → full 0/2/3 mix over 8,000 episodes), and training used `max_steps`=2000.

3.4 Detailed Description of Best Models

3.4.1 Approach 1: Recurrent PPO with LSTM (Best Score: -1447.55)

- **Algorithm**: Proximal Policy Optimization (PPO) with LSTM-based recurrent actor-critic
- **Architecture**: Linear(77→128) → ReLU → LSTM(128→128) → Actor head (128→5) + Critic head (128→1)
- **State representation**: 4-frame stack (72 dims) + previous action one-hot (5 dims) = 77 dims
- **Training**: Curriculum learning (spawn near box early, random later), dense reward shaping (sensor proximity, IR alignment, anti-circling), GAE (λ=0.95), entropy regularization (β=0.02), PPO clipping (ϵ=0.2)

- **Inference**: Greedy policy (argmax on actor logits). No heuristic fallback — the LSTM's memory enables learned temporal strategies for the search phase.

3.4.2 Approach 7: D3QN + PBRS (Best Score: -1540.74)

- **Algorithm**: Dueling Double DQN with PER and n-step returns
- **Architecture**: Dueling MLP (108→256→128, then split into value head 128→64→1 and advantage head 128→64→5). $Q(s, a) = V(s) + A(s, a) - \text{mean}(A)$
- **State representation**: 6-frame stack (108 dims)
- **Exploration**: Forward-biased ϵ-greedy with slow decay over 200K steps
- **Reward shaping**: 3-component PBRS (approach, heading, push) with scale 10.0 (heading $\times 0.5$, push $\times 2.0$)
- **Curriculum**: 4-phase progressive difficulty introduction

4 Experiments and Implementation Details

4.1 Training Setup

All experiments were conducted on a CPU-based environment (as required by the submission constraints) with Python 3 and PyTorch. All inference codes were designed to run on CPU with `map_location=''cpu''`.

4.2 Hyperparameter Summary

Parameter	Approach 1	Approach 2	Approach 3	Approach 4	Approach 5	Approach 6	Approach 7
Learning rate	3e-4	1e-4	3e-4	2e-4	2e-4	1e-4	1e-4
γ	0.99	0.99	0.99	0.99	0.99	0.99	0.99
Batch size	–	64	128	64	64	64	64
Buffer size	–	75K	75K	50K	75K	75K	50K
ϵ decay	–	300K	–	100K	500K	500K	200K
Target update	–	1K	Soft (τ=0.005)	1K	2K	2K	2K
Episodes	8K	8K	8K	5K	15K	12K	8K
Max steps	1K	1K	1K	1K	1K	1K	2K
Input dim	77	77	77	77	77	77	108
Frame stack	4	4	4	4	4	4	6
Prev. action	Yes	Yes	Yes	Yes	Yes	Yes	No
Recurrence	LSTM	GRU	GRU	No	No	GRU	No
Curriculum	Yes	Yes	Yes	Yes	No	No	Yes
PER	No	Yes	No	No	No	No	Yes
n-step	–	3	1	1	1	1	3

Table 2: Hyperparameter comparison across Approaches 1–7 (in order). "–" indicates not applicable.

4.3 Multi-Difficulty Training

For the **no-curriculum approaches** (Approaches 5 and 6), this weighted multi-difficulty sampling was used from the very beginning of training. In contrast, the **curriculum-based approaches** (Approaches 1–4 and 7) used staged progression (easier settings first, then harder conditions such as walls/blinking/full mixed difficulty), with Approach 7 ending in the same mixed 0/2/3 setting for final robustness.

4.4 Local Evaluation

To reduce overfitting to specific seeds, local evaluation was performed with multiple random seeds per difficulty level, following the Codabench evaluation protocol. I used the custom `rank_checkpoints_codabench.py` script to automate checkpoint scoring and comparison.

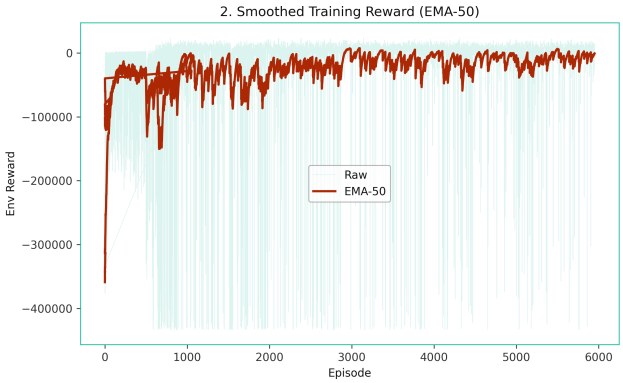
5 Results

5.1 Codabench Test Phase Results

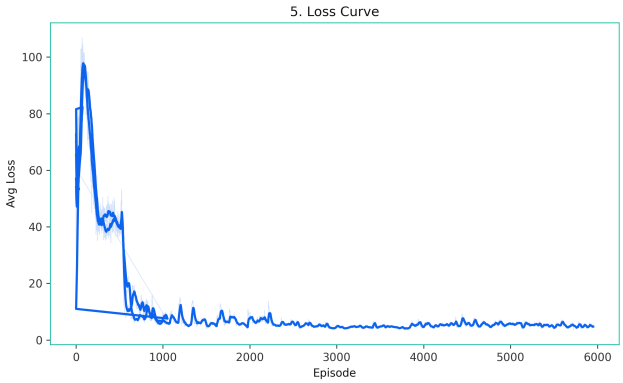
#	Method	Best Checkpoint	Best Score	Worst Score
1	Recurrent PPO (LSTM)	ep7200	−1447.55	−387621
7	Refined D3QN + PBRS	ep05200	−1540.74	−265222
3	Discrete SAC (GRU)	ep2000	−1655.64	−381539
6	D3QN v2, No Curriculum	D3QN_FINAL_V1	−9435.41	−326035
2	D3QN + GRU + PER	ep4000	−25715.40	−378919
5	Simple DDQN + Heuristic	ep500	−156899.23	−318887

Table 3: Best Codabench score per approach on the final test phase (sorted by best score, higher is better). Scores are mean cumulative reward across 3 difficulty levels × 10 runs, with wall obstacles. Approach 4 (Hybrid Heuristic) was not submitted to the final phase as it performed immensely bad on *Local Evaluation*.

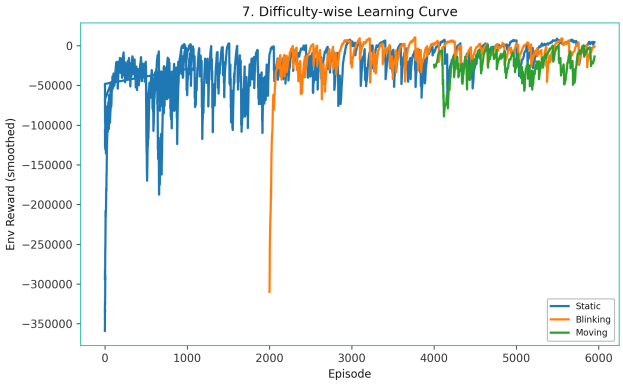
5.2 Training Curves (Approach 7 – Refined D3QN with PBRS)



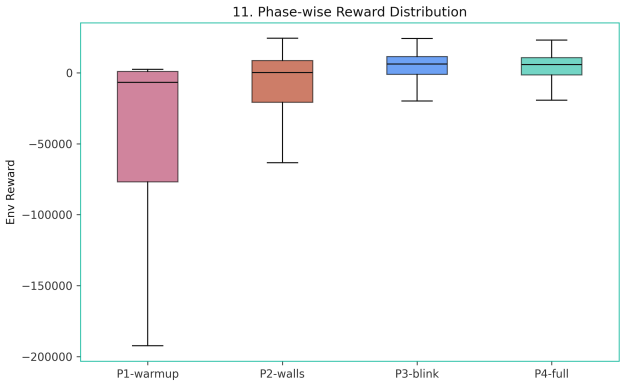
(a) Smoothed training reward (EMA-50). Reward improves sharply after curriculum Phase 1, then stabilizes with high variance.



(b) TD loss curve. Loss spikes during early episodes due to large reward magnitudes, then converges to a low stable value by episode 2000.



(c) Per-difficulty learning curves. Static (Level 0) learns fastest; blinking and moving levels introduced later via curriculum show slower improvement.



(d) Reward distribution per curriculum phase. Variance decreases and median reward improves across phases, indicating progressive learning.

Figure 1: Training diagnostics for Approach 7 (Refined D3QN with PBRS, 6000 episodes). All plots use true environment reward (not shaped reward).

5.3 Codabench Score Analysis

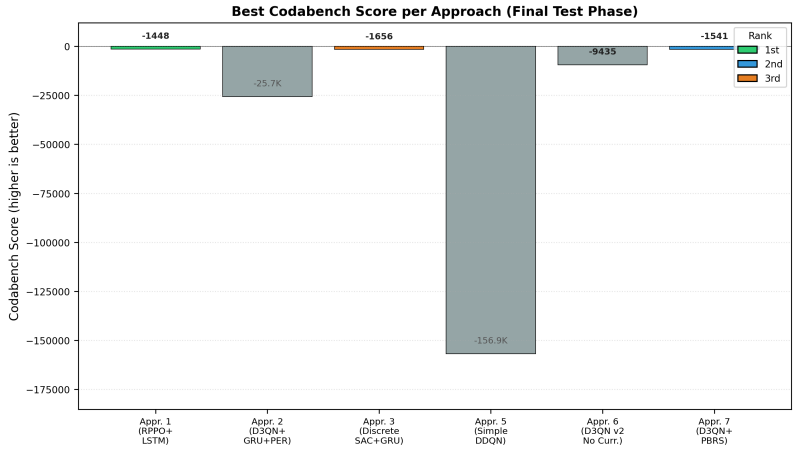


Figure 2: Best Codabench score per approach on the final test phase. Approaches 1 (RPPO+LSTM) and 7 (D3QN+PBRS) significantly outperformed all others. Approach 4 (Hybrid) was not submitted to this phase.

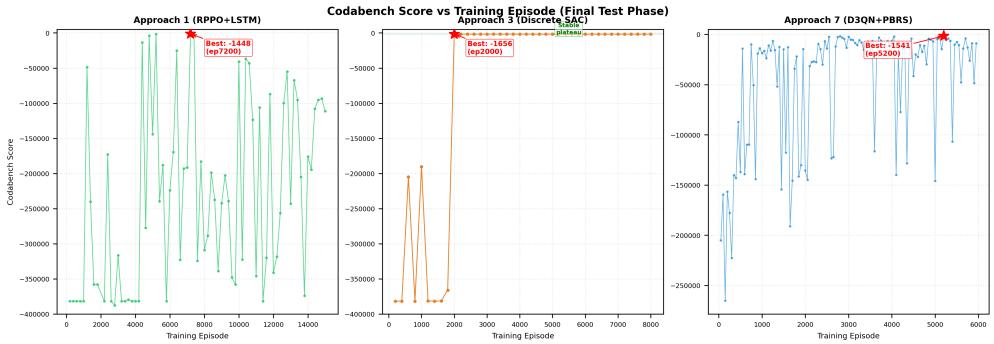


Figure 3: Codabench score as a function of training episode checkpoint for the three top-performing approaches. Approach 1 (RPPO) shows extreme sensitivity to checkpoint selection, Approach 3 (Discrete SAC) converges to a stable plateau after ep2000, and Approach 7 (D3QN+PBRS) exhibits high variance throughout training.

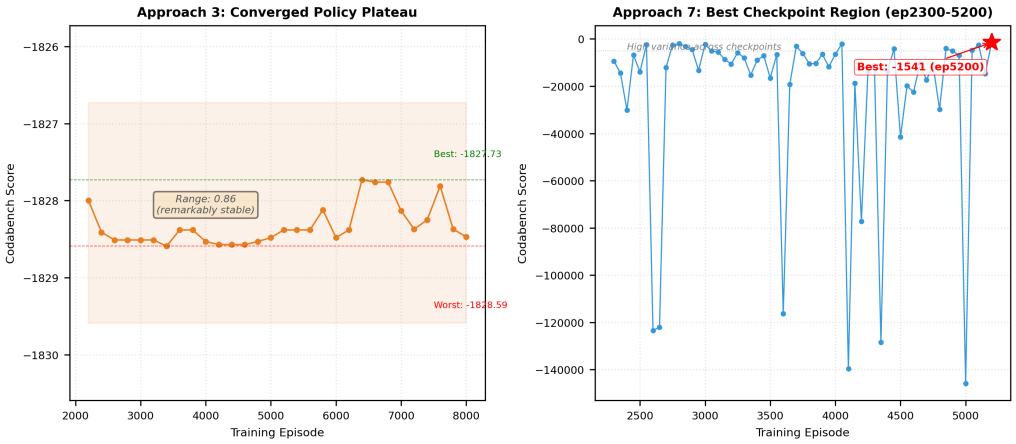


Figure 4: Zoomed analysis: (Left) Approach 3's converged plateau (ep2200+) spans only ∼0.86 points (−1827.73 to −1828.59), indicating a stable but suboptimal local minimum. (Right) Approach 7's best region (ep2300–5200) shows extreme variance, with the best checkpoint at ep5200 (−1541) surrounded by checkpoints scoring 10×–100× worse.

5.4 Key Observations from Results

Common behavior across all agents. Visual inspection of rollouts showed that all approaches—including the best-scoring ones—default to repetitive rotation when no sensors are active. A pure rotation policy scores roughly -1980 (idle penalty over 2000 steps). The score differences between approaches (-1447 to -1980) come from how well each agent responds *when sensors do activate* on seeds where the box happens to spawn nearby. On most seeds, every agent rotates for the full episode.

Method-level observations:

- **Approach 1 (RPPO + LSTM, -1447)**: Best overall. The LSTM retains temporal context, and PPO's entropy regularization keeps the policy from collapsing too early. On favorable seeds it approaches, attaches, and pushes reliably. *Weakness*: still rotates when sensors are zero; highly sensitive to checkpoint selection (ep7200 scores -1447, ep15000 scores much worse due to overfitting).
- **Approach 7 (D3QN + PBRS, -1541)**: Second best. PBRS gives a principled shaping signal and curriculum builds solid approach/push behavior. *Weakness*: extreme variance across checkpoints (-1541 to -156668)—most checkpoints are bad, only a few happen to work.
- **Approach 3 (Discrete SAC, -1656 to -1829)**: Most stable—nearly all checkpoints from ep2200 onward score within a $\sim$0.86-point range. Entropy regularization prevents total collapse. *Weakness*: converges to a fixed local optimum; the sensor-response behavior is weaker than Approaches 1 and 7, so it gains less from favorable seeds.
- **Approach 2 (D3QN + GRU + PER, -25715)**: This model had useful components (memory, prioritized replay, and n-step returns), and I trained it again in Phase 4 with sufficient compute, but it still did not work well. After many repeated all-zero observations, the GRU state became less useful, and the value updates could not recover a reliable policy for reacting to sensor signals. *Strength*: it reached the best single-episode score in Phase 3 (-1515), but that did not become stable performance on Codabench.
- **Approach 6 (D3QN v2, no curriculum, -9435)**: Without curriculum, it never got enough near-box experience to learn approach/push behavior. GRU hidden state reset helped avoid saturation but was not enough. *Strength*: simpler setup, no curriculum dependency.
- **Approach 5 (DDQN + heuristic, -156899)**: The lawnmower heuristic was a good idea for search but did not work well in practice with walls. The MLP had no recurrence, so it could not maintain temporal context. Only one submission was made.

Curriculum learning improved sensor-conditioned control, but not box search under zero information. Approaches 1 and 7 (with curriculum) scored -1447 and -1541, while Approaches 5 and 6 (without curriculum) scored -156899 and -9435. The main benefit of curriculum was it repeatedly exposed the agent to near-box states, so it learned better *approach/attach/push* behavior once sensors became active. However, it did not solve the harder *search* problem: when all sensor bits are zero, the policy still has no directional signal and often falls back to rotation. Phase 3's weaker results (-1500 to -4000) were mainly due to under-training (max_steps = 1000, stopped at $\sim$2K episodes), not a flaw in curriculum itself—Phase 4's longer training improved this.

5.5 Learning Dynamics

Best checkpoints appeared around episodes 5000–8000. Early episodes are too random, and later episodes tend to overfit to curriculum or shaped rewards. Approach 1's ep7200 outperformed ep15000, confirming that more training is not always better.

6 Error Analysis and Discussion

6.1 Common Failure Modes

The main failure is that when all 18 sensor bits are zero (box out of range), no approach learned to move toward the box—they all rotate in place. This means the task splits into two parts: (1) **search** (finding the box with no sensor information), which no method solved, and (2) **exploit** (acting on sensor signals once the box is detected), which curriculum-trained methods handled well.

Per difficulty level: **Level 1** fails when the box is far away (agent never detects it). **Level 2** adds blinking, so the agent can lose the signal mid-approach and revert to rotation. **Level 3** adds box movement, making accidental detection even rarer. **Walls** add -200 collision penalties on top of the idle penalty.

6.2 Why Score Differences Exist

All agents share the same rotation behavior, but scores differ because: (1) curriculum-trained agents (1, 7) respond better when sensors activate, gaining more reward from favorable seeds; and (2) the 30-seed average is sensitive to a few good seeds—one successful push episode can improve the mean by hundreds of points.

6.3 Limitations and Future Improvements

The biggest unsolved problem is *search*: when all 18 sensor bits are zero, the agent has no information about where the box is, so it just rotates in place. None of the seven approaches solved this. Due to limited time, I could not explore the following directions, but they are promising for future work:

- **Intrinsic motivation** [8, 9]: Right now the agent only gets reward when it interacts with the box. With curiosity-based bonuses (like ICM or RND), the agent would also get a small reward for visiting new positions it has not seen before. This would push it to move around the arena instead of staying in one spot, even when sensors show nothing.
- **Hierarchical RL** [10]: Instead of one policy trying to do both search and push, we could use two separate controllers—a high-level one that decides "search for the box" vs. "go push the box," and a low-level one that executes each sub-task. This way each controller can specialize in its own problem.
- **Imitation learning** [11]: We could first collect demonstrations from a hand-coded or human-controlled agent that searches the arena effectively (e.g., moving in a systematic pattern). Then use behavioral cloning or DAgger to train the agent to copy this behavior before fine-tuning with RL. This gives the agent a good starting policy instead of learning from scratch.
- **RLHF (RL from Human Feedback)** [12]: Instead of designing a reward function manually, a human could watch agent trajectories and label which ones show good exploration vs. just rotating. A reward model trained on these labels would then guide the agent toward exploration behavior that is hard to specify with a simple formula.
- **Systematic search patterns**: The simplest solution to this problem—hard-code a lawnmower or spiral movement pattern for when all sensors are zero, and only switch to the learned RL policy when sensors activate. This is not learned behavior, but it directly solves the "no information means no movement" problem.
- **Richer observations**: The current 18-bit binary observation is very limited—either a sensor fires or it does not. Continuous sensors (e.g., distance and angle to the box) would give the agent a gradient signal even when the box is far away. Adding positional history would also help the agent remember where it has already been, so it avoids revisiting the same spots.
- **Domain randomization** [13]: Training with randomized arena configurations (different spawn locations, wall layouts, box speeds) could make the policy more robust across different seeds and difficulty levels, reducing the high variance observed in evaluation.

7 Conclusion

This project showed that the OBELIX task splits into two problems of very different difficulty: *search* (finding the box) and *exploit* (acting once the box is detected).

1. **Search remains unsolved**: All seven approaches default to rotation when sensors are inactive. The binary observation space gives no useful signal for directed movement.
2. **Curriculum learning solves exploit**: With enough training (Phase 4), curriculum-based approaches (1 and 7) learned reliable approach/attach/push behavior, which is why they score better on seeds where the box spawns nearby.
3. **Scores reflect sensor-response quality**: The difference between -1447 and -1980 comes from how well each agent uses favorable seeds and how well it pushes the box to boundary when seen, not from better search.
4. **Checkpoint selection**: Best results come from mid-training checkpoints (ep5000–8000); later ones overfit.

The main takeaway: in this POMDP, the bottleneck is search, not exploitation. Curriculum learning with sufficient compute teaches the agent what to do once it finds the box, but finding the box in the first place remains the open challenge. If done any, future work should focus on intrinsic motivation, hierarchical RL, and richer state representations for a better search methodology.

Project Links

- **YouTube Demo:** `https://youtu.be/5Z8gjNmatkY?si=EvXoElV98xMPQ18Y`
- **GitHub Repository:** `https://github.com/AritraAmbudhDutta/CS780-OBELIX`

LLM Usage Declaration

- Which Large Language Model (LLM) did you use to complete the competition or write the report?
  Github Copilot was used in a limited advisory capacity.

- In which part did you use the LLM, and how did it help you?

Part	Used LLM? (Yes/No)	Purpose of Using LLM
Understanding concepts	Yes	Clarifying PBRS theory, DRQN architecture details
Ideation / brainstorming	Yes	Generating list of possible approaches when stuck
Debugging small code snippets	No	
Plotting / visualization code	Yes	Used LLM help to write the complex plot codes
Grammar / writing style	No	
Report structure / section ideas	Yes	ideas about how to organize content into required sections
Other (specify)	No	

- Did you refer to any other sources or websites apart from the LLM and lecture slides?
  Yes: (1) Sutton & Barto textbook [14], (2) Original OBELIX paper by Mahadevan & Connell [1], (3) Double DQN paper [4], (4) Dueling DQN paper [3], (5) PPO paper [2], (6) PER paper [5], (7) PBRS paper by Ng et al. [7], (8) DRQN paper by Hausknecht & Stone [15].

- Apart from any LLM usage, what was your own contribution?
  All algorithmic implementation, training, hyperparameter tuning, debugging, and experimental evaluation were performed independently. The core contributions include: (1) implementing and training 7 distinct RL approaches from scratch using PyTorch based on lecture pseudocode and paper descriptions; (2) designing the heuristic fallback search policy (lawnmower pattern) based on manual play observations; (3) conducting systematic root cause analysis of the circling failure mode; (4) developing the multi-difficulty training strategy; (5) building automated checkpoint ranking and submission pipelines; (6) all experimental analysis and conclusions drawn from the results.

References

[1] S. Mahadevan and J. Connell, "Automatic programming of behavior-based robots using reinforcement learning," *Artificial Intelligence*, vol. 55, no. 2-3, pp. 311–365, 1992.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[3] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling network architectures for deep reinforcement learning," *Proceedings of the 33rd International Conference on Machine Learning*, pp. 1995–2003, 2016.

[4] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, no. 1, 2016.

[5] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016.

[6] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," *Proceedings of the 35th International Conference on Machine Learning*, pp. 1861–1870, 2018.

[7] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," *Proceedings of the 16th International Conference on Machine Learning*, pp. 278–287, 1999.

[8] D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell, "Curiosity-driven exploration by self-supervised prediction," in *Proceedings of the 34th International Conference on Machine Learning*, pp. 2778–2787, 2017.

[9] Y. Burda, H. Edwards, A. Storkey, and O. Klimov, "Exploration by random network distillation," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019.

[10] A. S. Vezhnevets, S. Osindero, T. Schaul, N. Heess, M. Jaderberg, D. Silver, and K. Kavukcuoglu, "FeUdal networks for hierarchical reinforcement learning," in *Proceedings of the 34th International Conference on Machine Learning*, pp. 3540–3549, 2017.

[11] S. Ross, G. J. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics*, pp. 627–635, 2011.

[12] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei, "Deep reinforcement learning from human preferences," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[13] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 23–30, 2017.

[14] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. MIT Press, 2nd ed., 2018.

[15] M. Hausknecht and P. Stone, "Deep recurrent q-learning for partially observable MDPs," *AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents*, 2015.